МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Руководитель
программы _____ Ш.Г. Магомедов

«30» мая 2025 г.

ИТОГОВАЯ АТТЕСТАЦИОННАЯ РАБОТА

по дополнительной программы профессиональной переподготовки
«Программные средства решения прикладных задач искусственного интеллекта»

На тему: «Модель распознавания вариант 300»

Обучающийся _____ Калинин Никита Викторович
Подпись *Фамилия, имя, отчество*

группа ЭФБО-09-23

Руководитель работы _____ Ш.Г. Магомедов
подпись

Москва 2025 г.

ФИО: Калинин Никита Викторович
Группа: ЭФБО-09-23
Тема: «Обучение модели тематической классификации сообщений из Telegram-каналов на основе контекстной языковой модели RuBERT»

СОДЕРЖАНИЕ

Содержание 3
Введение 4
1 Аналитический обзор 7
1.1 Анализ предметной области 7
1.2 Анализ аналогов разрабатываемого решения 8
1.3 Описание используемых данных 10
2 Проектная часть 27
2.1 Постановка задачи обучения модели 27
2.2 Выбор и обоснование архитектуры 27
2.3Процесс обучения 28
2.4 Проведение экспериментов и анализ результатов 29
ЗАКЛЮЧЕНИЕ 31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 33
ПРИЛОЖЕНИЯ 36
Приложение А – Код из исследовательского анализа данных 36
Приложение Б – Исходный код скрипта парсинга Telegram-каналов 40
Приложение В – Исходный код обучения модели 42

ВВЕДЕНИЕ

В последние годы наблюдается стремительный рост числа Telegram-каналов, публикующих разнообразный контент, ориентированный на широкую аудиторию. Пользователи, подписываясь на такие каналы, зачастую сталкиваются с проблемой информационной перегрузки: среди большого потока сообщений лишь небольшая часть действительно соответствует их интересам. Это приводит к снижению эффективности потребления информации и усложняет навигацию в личных Telegram-лентах.

Одним из возможных решений данной проблемы является автоматическая фильтрация контента по тематикам, представляющим интерес для конкретного пользователя. Для реализации такого подхода необходим инструмент, способный анализировать и классифицировать сообщения, исходя из их содержания.

В рамках данной работы был собран и размечен датасет, включающий в себя посты из различных Telegram-каналов, охватывающих широкий спектр тематик. Каждый пост был отнесён к одному из заранее определённых тематических классов. На основе полученного датасета была разработана и обучена модель машинного обучения, способная автоматически определять тематику новых сообщений. Модель позволяет классифицировать входящий контент и, таким образом, предоставляет возможность пользователю фильтровать сообщения, оставляя только те, которые соответствуют его интересам.

Актуальность темы

Современное общество характеризуется стремительным ростом объёмов информационных потоков, что особенно заметно на цифровых коммуникационных платформах, таких как Telegram. Ежедневно пользователи сталкиваются с огромным количеством сообщений из множества различных каналов, что приводит к явлению информационной перегрузки. В условиях избыточного объёма информации человеку становится сложно эффективно фильтровать и выделять действительно важные и интересующие его сообщения.

Автоматическая классификация и фильтрация контента становится всё более востребованным инструментом для повышения удобства и эффективности взаимодействия с информацией. Это особенно актуально для Telegram-каналов, охватывающих широкий спектр тематик — от новостей и аналитики до развлечений и специализированных профессиональных сообществ.

Создание моделей, способных автоматически определять тематику сообщений и адаптироваться под индивидуальные интересы пользователей, позволяет значительно улучшить качество потребления информации и сократить затраты времени на её обработку. Таким образом, разработка и внедрение таких моделей способствует решению одной из ключевых задач современного информационного общества — преодолению информационного шума и повышению релевантности получаемого контента.

Цель работы: разработка модели машинного обучения для автоматической классификации контента Telegram-каналов по тематическим категориям.

Задачи работы

1. Провести анализ представленного датасета с целью выявления особенностей распределения данных и предварительной подготовки текста;
2. Разработать модель классификации, основываясь на результатах анализа и особенностях структуры текстового контента;
3. Провести серию экспериментов с анализом качества модели.

Объектом данного исследования являются текстовые сообщения, публикуемые в каналах мессенджера Telegram, как тип информационного контента, подлежащего автоматизированной обработке и классификации.

Предметом исследования выступают методы и алгоритмы машинного обучения, применяемые для автоматической классификации текстовых данных по заранее определённым тематическим категориям.

В качестве основных методов исследования используются методы системного анализа и синтеза при изучении предметной области, сравнительный

анализ аналогов, моделирование данных, проектирование баз данных и интерфейсов, а также тестирование программных модулей.

Информационной базой исследования являются открытые источники, в том числе доступные в сети Интернет.

Структура работы

Данная работа состоит из введения, трёх основных разделов — аналитического обзора, проектной части и заключения, а также списка литературы и приложений.

Во введении расписана актуальность выбранной темы, сформулированы цель и задачи исследования, определены объект и предмет работы. Также представлено краткое описание структуры работы.

Аналитический обзор включает обзор предметной области, анализ существующих решений и моделей, применяемых для классификации текстового контента, а также описание подготовленного датасета, его структуры и особенностей.

В проектной части описывается постановка задачи обучения модели, выбор и обоснование её архитектуры, этапы подготовки данных, процесс обучения, проведённые эксперименты и анализ полученных результатов.

Заключение содержит выводы по проделанной работе, оценку степени достижения поставленных целей, а также предложения по возможным направлениям дальнейших исследований.

В списке литературы приведены источники, использованные при написании работы, включающие как теоретические материалы по машинному обучению и обработке естественного языка, так и практические исследования в данной области.

1 АНАЛИТИЧЕСКИЙ ОБЗОР

1.1 Анализ предметной области

Автоматическая классификация текстов является одной из ключевых задач в области обработки естественного языка (NLP). Она заключается в отнесении текстового сообщения к одной из заранее определённых категорий на основе его содержания. Такие методы широко применяются для фильтрации спама, анализа пользовательских отзывов, распределения новостных сообщений по рубрикам, а также в системах рекомендаций и интеллектуальных помощниках.

С развитием технологий всё большее распространение получают модели на основе глубинного обучения. Одним из наиболее эффективных подходов стало использование трансформеров, в частности архитектуры BERT (Bidirectional Encoder Representations from Transformers), предложенной компанией Google. Эта модель обучается на больших корпусах текстов и способна учитывать контекст слов с обеих сторон, что делает её особенно эффективной для задач классификации, извлечения сущностей, вопросов и ответов и других. Для работы с русскоязычными текстами была адаптирована модель RuBERT, разработанная исследовательским подразделением SberAI. Она представляет собой дообученную версию оригинального BERT на русскоязычных данных, включая Wikipedia и новостные статьи, что делает её особенно подходящей для анализа текстов на русском языке. Благодаря способности учитывать языковые особенности и контекстные связи между словами, RuBERT показывает высокие результаты в различных NLP-задачах, включая тематическую классификацию.

В контексте Telegram-сообщений задача усложняется рядом факторов: короткой длиной текстов, их неструктурированностью, использованием разговорной лексики, эмодзи и ссылок. Несмотря на это, модели на базе RuBERT демонстрируют высокую устойчивость к таким особенностям и способны эффективно выделять релевантные признаки даже из кратких сообщений.

Таким образом, применение RuBERT в задаче классификации Telegram-постов представляется обоснованным и перспективным решением, соответствующим современному уровню развития технологий в области обработки текстовой информации.

1.2 Анализ аналогов разрабатываемого решения

В последние годы наблюдается активное развитие инструментов и моделей для классификации и анализа контента в Telegram-каналах. Рассмотрим некоторые из них:

1. pytopicgram — библиотека на Python, предназначенная для извлечения данных и тематического моделирования из сообщений Telegram-каналов. Она предоставляет удобные средства для сбора сообщений, анализа вовлечённости аудитории и идентификации обсуждаемых тем с использованием передовых методов моделирования. Эта библиотека упрощает процесс анализа и позволяет исследователям изучать, как распространяется контент и как аудитория взаимодействует с ним на платформе Telegram [16];

2. Tonfor.pro — веб-сайт, предоставляющий доступ к ресурсам и инструментам, связанным с блокчейном The Open Network (TON). Проект выполняет функции агрегатора ссылок, визуализации транзакций и автоматизированного анализа сообщений из Telegram-каналов. Система использует языковую модель Llama 3.3-70B для анализа обсуждаемых тем, оценки рыночного настроения и извлечения ключевых факторов [17].

Таблица 1.1 – Сравнение аналогов разрабатываемого решения

№	Название системы	Основная цель	Используемая технология	Производительность и ресурсы	Доступность исходного кода
1	pytopicgram	Тематическое моделирование сообщений Telegram	Python, тематическое моделирование	Средняя нагрузка на CPU, требует достаточного объёма оперативной памяти для обработки больших объёмов данных.	Открытый репозиторий на GitHub.
2	Tonfor.pro	Анализ контента Telegram-каналов в контексте TON	Llama 3.3-70B, визуализация данных	Использует Llama 3.3-70B, требующую значительных вычислительных ресурсов и памяти. Рекомендуется наличие GPU для оптимальной работы.	Закрытый код.

Сравнительный анализ показывает, что существующие решения в основном ориентированы на анализ активности и вовлечённости аудитории, а также на тематическое моделирование в контексте определённых экосистем. Однако они не всегда предоставляют возможность точной классификации контента Telegram-каналов по заранее определённым тематическим категориям.

Всё это подчёркивает актуальность создания специализированной, гибкой и доступной системы анализа контента Telegram-каналов, адаптированной под реальные потребности пользователей. Такое решение позволит автоматически классифицировать сообщения по тематическим категориям, повысить точность анализа публикаций, а также обеспечить более высокую релевантность и структурированность потребляемого контента. Основа на предобученной языковой модели RuBERT делает систему универсальной и применимой к различным сферам, вне зависимости от тематической направленности.

1.3 Описание используемых данных

1.3.1 Подготовка данных

Для подготовки данных были выбраны несколько Telegram-каналов с различной тематикой сообщений и постов. Далее был написан скрипт, который использует Telegram User API для извлечения сообщений из Telegram-каналов (приложение Б.1). После было извлечено по несколько тысяч сообщений с каждого канала и сформирован первоначальный датасет. После этого была проведена разметка сообщений по следующим категориям:

1. other: тексты, не подходящие ни под одну из остальных категорий;
2. business: новости и информация о бизнесе;
3. finances: тексты о финансах, рынках, экономике, деньгах;
4. political: новости о политике, законах, выборах, государственных органах;
5. personal: личные происшествия, необычные или курьезные случаи;
6. stuff: развлекательные темы, праздники, гороскопы, мистические истории;
7. gaming: новости про видеоигры, киберспорт, игровые события;
8. science: наука, технологии, медицина, исследования;
9. moscow: новости о Москве;
10. weather: новости о погоде — прогнозы, катаклизмы;
11. it: новости IT и технологий — софт, железо, устройства, языки программирования, машинное обучение;
12. advertisement: рекламные тексты о товарах, услугах, акциях.

1.3.2 Описание подготовленных данных

	message_id	sender_id	text	date	channel	category
0	126168	-1001754252633	⚡Военнослужащие ВС Сирии **взяли под контроль...	2024-12-04 08:57:11+00:00	Топор Live	political
1	126166	-1001754252633	⚡Ученики новосибирской школы **баррикадируют ...	2024-12-04 08:51:11+00:00	Топор Live	personal
2	126165	-1001754252633	⚡Германия не исключает **введения своих миро...	2024-12-04 08:45:06+00:00	Топор Live	political
3	126164	-1001754252633	Доллар упал ниже 104₽, евро — ниже 110.\n\n[📈T...	2024-12-04 08:35:11+00:00	Топор Live	finances
4	126163	-1001754252633	⚡Учения ОДКБ в 2025 году пройдут в **Беларуси...	2024-12-04 08:28:45+00:00	Топор Live	political

Рисунок 1.1 — Первые 5 строк датасета

Как видно на рисунке 1.1, для работы с текстовыми данными потребуется предварительная обработка, включающая нормализацию текста (например, стемминг или лемматизацию), удаление стоп-слов, а также векторизацию с использованием методов, таких как TF-IDF или Word2Vec.

```
Размер датасета: 22099 постов
Колонки: ['message_id', 'sender_id', 'text', 'date', 'channel', 'category']
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22099 entries, 0 to 22098
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   message_id  22099 non-null  int64
 1   sender_id   22099 non-null  int64
 2   text        22099 non-null  object
 3   date        22099 non-null  datetime64[ns, UTC]
 4   channel     22099 non-null  object
 5   category    22099 non-null  object
dtypes: datetime64[ns, UTC](1), int64(2), object(3)
memory usage: 1.0+ MB
```

Рисунок 1.2 — Статистика датасета

На рисунке 2.1 представлена сводная статистика по датасету, содержащему 22 099 постов. Данные имеют чёткую структуру без пропущенных значений — все 6 колонок полностью заполнены. Ключевые поля включают:

1 text — основной текст поста, служащий входными данными для NLP-анализа;

2 date — временная метка в формате datetime64[ns, UTC], позволяющая изучать динамику публикаций;

3 category — целевая переменная, отражающая тематическую принадлежность поста.

Типы данных соответствуют ожидаемым: datetime64[ns, UTC] для даты, int64 для числовых идентификаторов и object для текстовых и категориальных значений. Это позволяет приступить к анализу без необходимости устранения пропусков.

Объём выборки достаточен для проведения статистически обоснованных выводов. Однако при обучении моделей машинного обучения может потребоваться дополнительная балансировка классов по категориям.

1.3.3 Анализ качества данных

```
Пропуски в данных:
message_id     0
sender_id      0
text           0
date           0
channel        0
category       0
text_length    0
dtype: int64

Полные дубликаты: 0
Дубликаты текста: 772
Дубликаты текста с категориями: 740
```

Рисунок 1.3 — Статистика пропусков и дубликатов

Как показано на рисунке 1.3, датасет обладает высокой целостностью — все столбцы полностью заполнены, что позволяет сразу приступить к содержательному анализу. Однако в ходе предварительной проверки были обнаружены текстовые дубликаты: 772 полностью идентичных сообщения, из которых 740 совпадают не только по тексту, но и по категории

Наличие таких дубликатов может негативно повлиять на обучение модели, формируя искажённое представление о значимости повторяющихся записей.

Чтобы избежать смещения результатов и переобучения, было принято решение удалить все дубликаты текста. Так же было принято решение удалить объекты с классом other, так как они являются выбросами при разметке.

Используемый код приведён в приложении А.1.

1.3.4 Анализ распределения постов по категориям

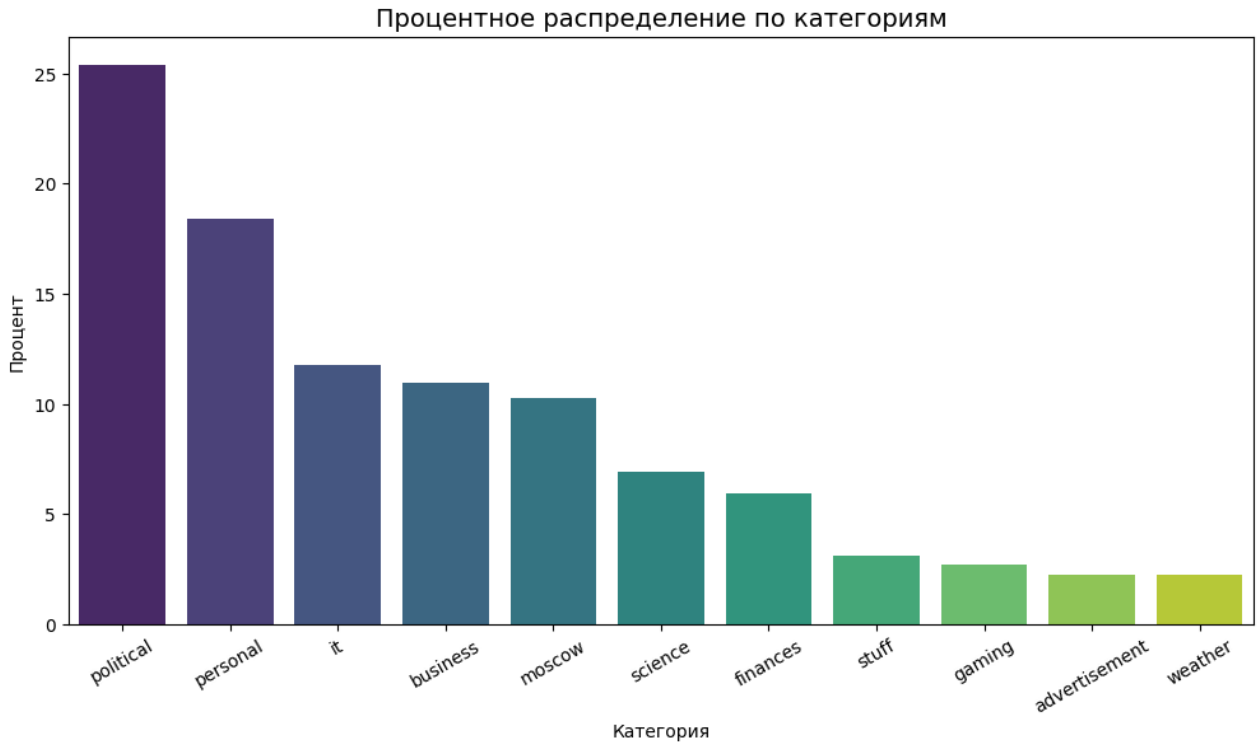


Рисунок 1.4 — График процентного распределения постов по категориям

На графике 1.4 представлено процентное распределение постов по категориям. Анализ этого распределения выявил существенный дисбаланс классов. Так, категория «Политика» охватывает более 20% всех записей, в то время как доля постов в категории «Реклама» не превышает 2.5%. Подобное несоответствие влияет на обучение моделей классификации, склоняя алгоритмы к предсказанию преобладающего класса ради оптимизации общей точности.

Для решения этой проблемы в дальнейшем планируется использовать стратифицированное разбиение данных и стратифицированную кросс-валидацию (StratifiedKFold) [1], а также оценивать качество моделей по F1-мере вместо accuracy [2][3], что позволит более объективно учитывать баланс между классами.

1.3.5 Анализ длины текста

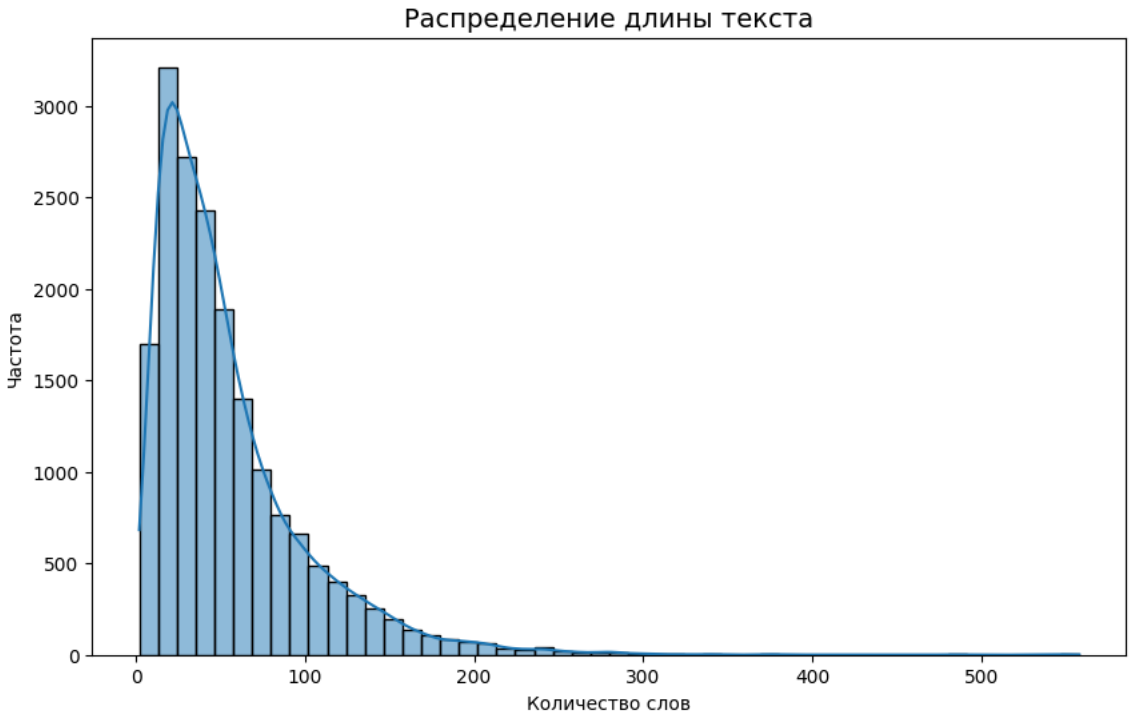


Рисунок 1.5 — График частотного распределения длинны текста

Из графика 1.5 можно увидеть, что средняя длина сообщения составляет около 54 слов, что указывает на относительно небольшой объём текстов, представленных в данном датасете.

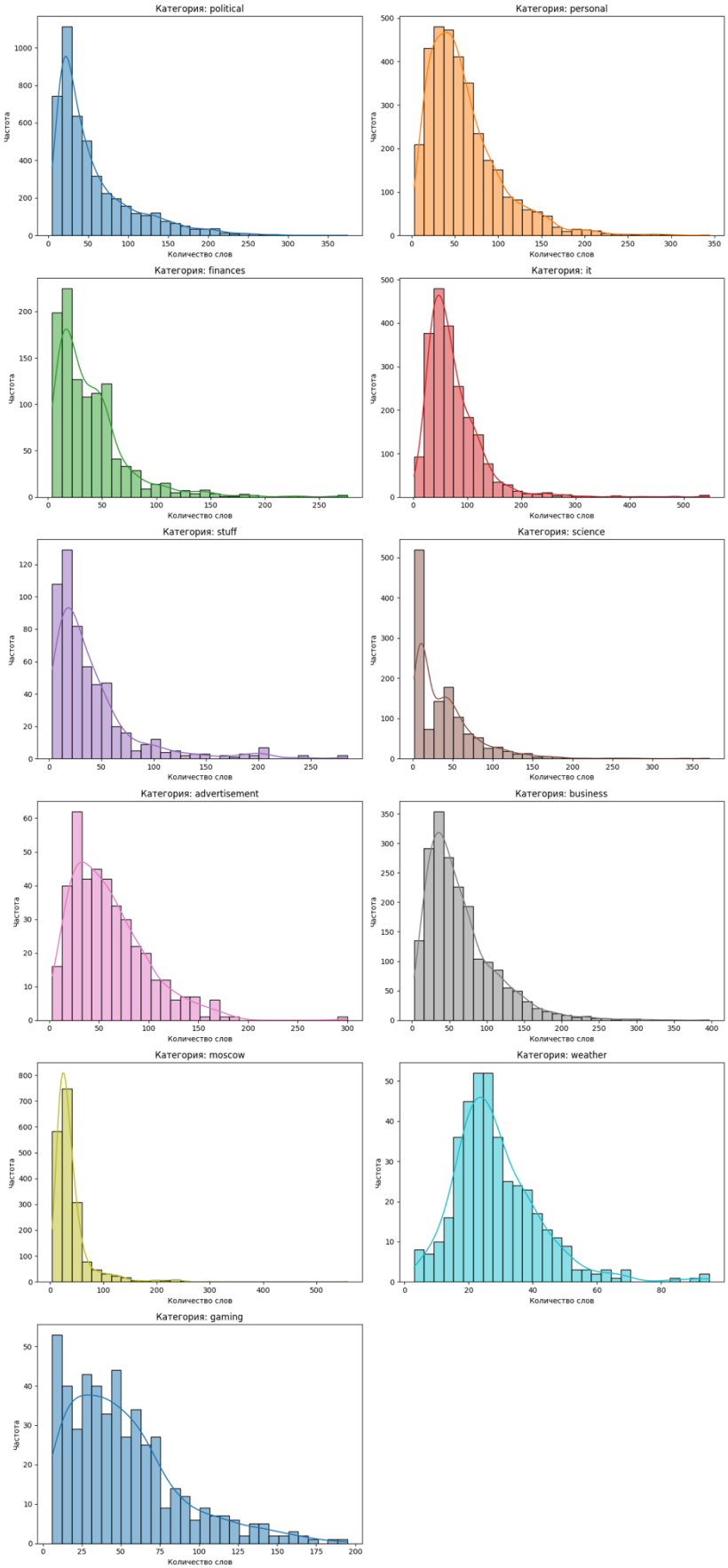


Рисунок 1.6 — Графики частотного распределения количества слов по категориям

Исследование длины сообщений по графику 1.6 выявило значительные различия между категориями. Тематика IT лидирует по объёму контента со средним значением 73,8 слова на пост, что на 45% превышает показатель аутсайдера — категории «Погода» с 28,6 слов. Медианные значения подтверждают эту тенденцию: от 61 слова в IT до 26 слов в сообщениях о погоде.

Прослеживается чёткая тематическая закономерность: технические и профессиональные категории, такие как IT и Бизнес, характеризуются наиболее развёрнутыми сообщениями; личные и развлекательные темы занимают промежуточные позиции; утилитарные категории, включая Финансы и Погоду, отличаются максимальной лаконичностью.

Особого внимания заслуживают категория Политика, где при среднем показателе 55,8 слова медиана составляет лишь 38 слов, что свидетельствует о большом числе крайне коротких сообщений на фоне редких длинных; а также категория Реклама, демонстрирующая сбалансированные показатели (59,9 слова в среднем и 52 — медиана), указывающие на стандартизированный формат коммерческих постов.

1.3.6 Предобработка данных

После проведённого краткого анализа была выполнена предобработка данных, включающая удаление ссылок, хештегов, элементов markdown-разметки, эмодзи, а также удаление стоп-слов из списка библиотеки NLTK [4] и преобразование слов в лемматизированную форму с помощью библиотеки SpaCy [5][6] (приложение А.2). Для упрощения работы с датасетом предобработанный текст был векторизован с использованием TF-IDF [9].

Данная обработка позволяет унифицировать данные для последующего NLP-анализа, устраняя технический «шум» без утраты смыслового содержания. Особое внимание уделялось удалению ссылок, поскольку их массовое присутствие могло искажать статистические показатели из-за повторяющихся элементов.

1.3.7 Анализ топа из 20 самых частых слов

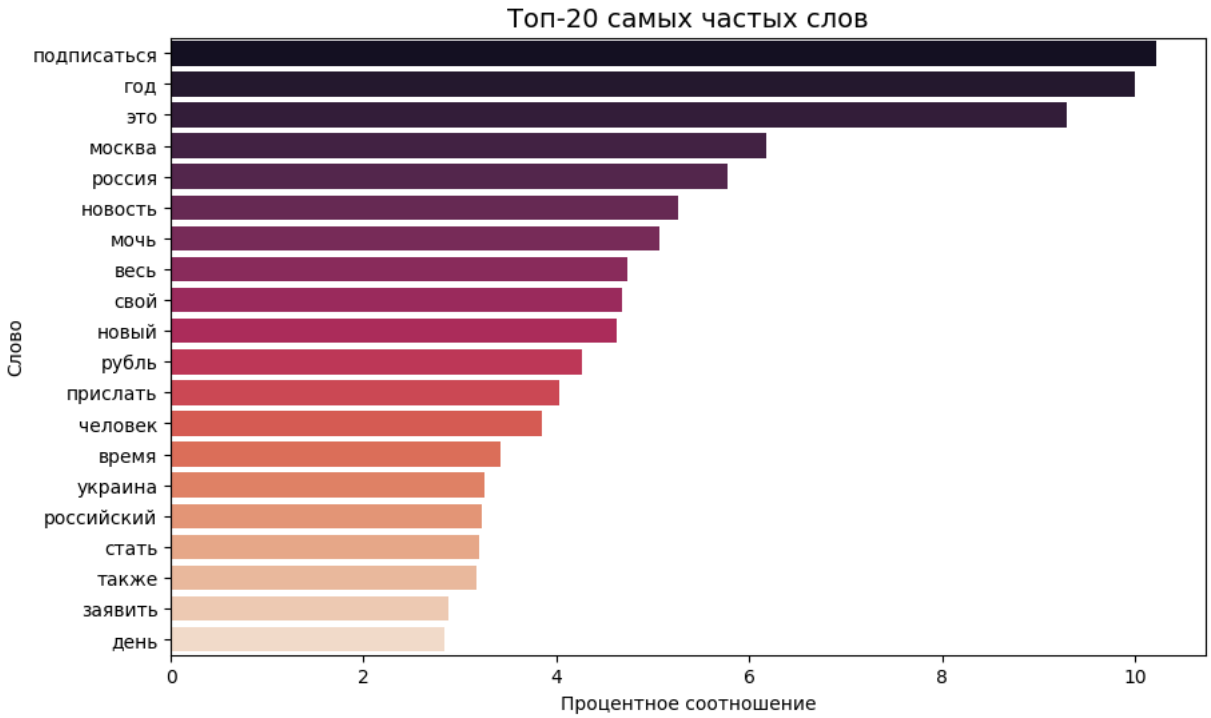


Рисунок 1.7 — График топ-20 самых частых слов в сообщениях

После удаления стоп-слов анализ частотности (рисунок 1.7) выявил характерные языковые паттерны. В текстах присутствуют организационные маркеры, такие как «подписаться» и «канал», которые были удалены как шаблонные элементы. Географические указатели, например «Россия» и «Москва», отражают локальную специфику, а тематические термины — «правительство», «компания» — указывают на ключевые темы. Глаголы действия, такие как «сообщать» и «писать», характерны для новостного стиля изложения. Для повышения качества данных была проведена дополнительная очистка от шаблонных фраз (приложение А.3).

1.3.8 Анализ топ-слов по категориям

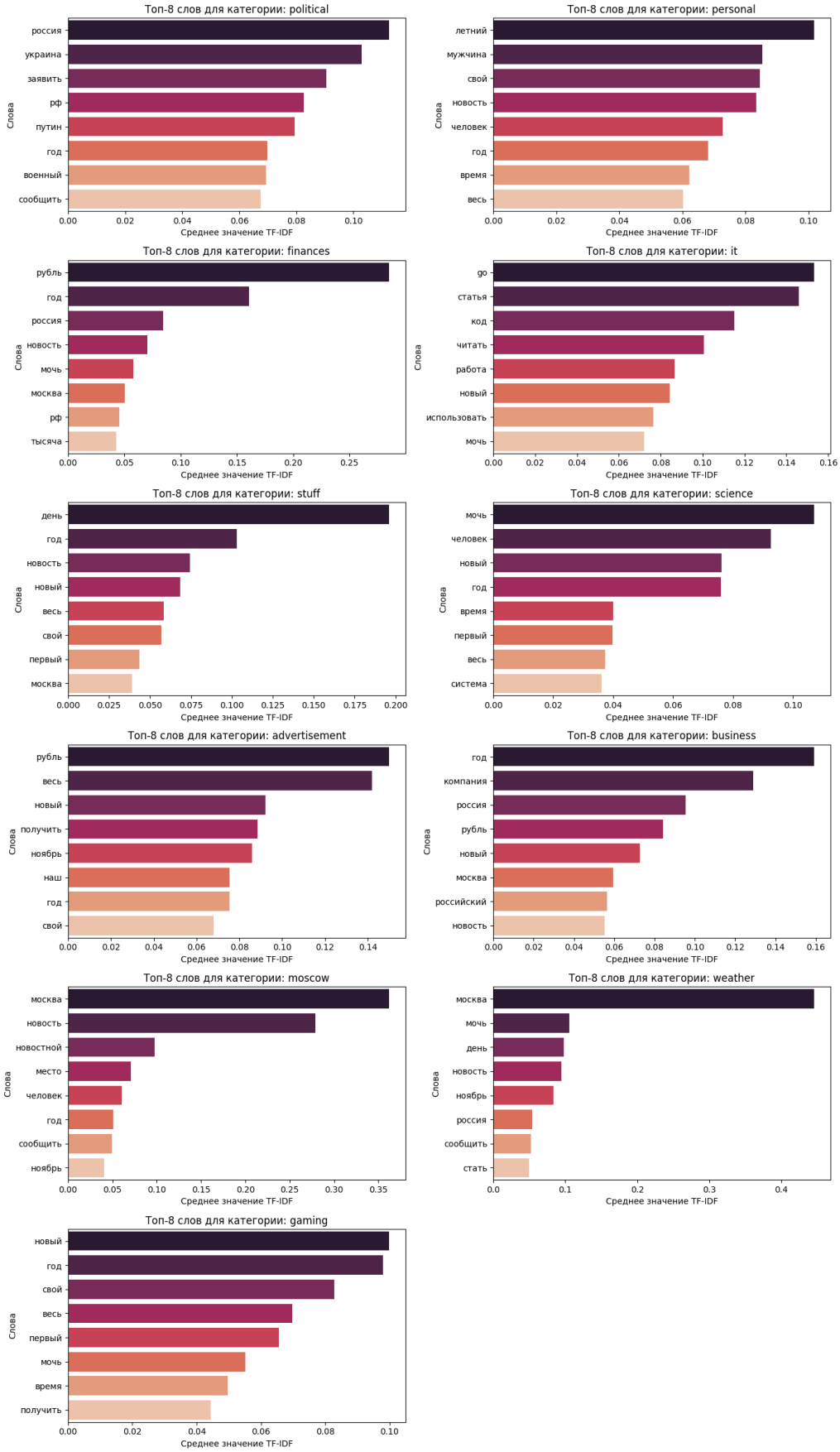


Рисунок 1.8 — График топ-слов по категориям

Анализ графика топ-слов по категориям (рисунок 1.8) показывает, что в разделе «Политика» доминируют термины, связанные с Россией и политическими событиями, тогда как в категории «Личное» преобладают слова, указывающие на людей. В «Финансах» часто встречаются упоминания о деньгах, а в «IT» — технические термины, такие как «python» и «данные». В категориях «Разное» и «Прочее» присутствуют слова, связанные с действиями и текущими событиями, например «сегодня» или «человек». Научные посты акцентируют внимание на времени и возможностях, рекламные — на деньгах и датах. В бизнес-контенте выделяются термины, связанные с компаниями и экономикой, а в постах о Москве преобладает географическая тематика. Таким образом, каждое ключевое слово органично соответствует тематике своей категории, отражая особенности соответствующей области.

1.3.9 Анализ биграмм

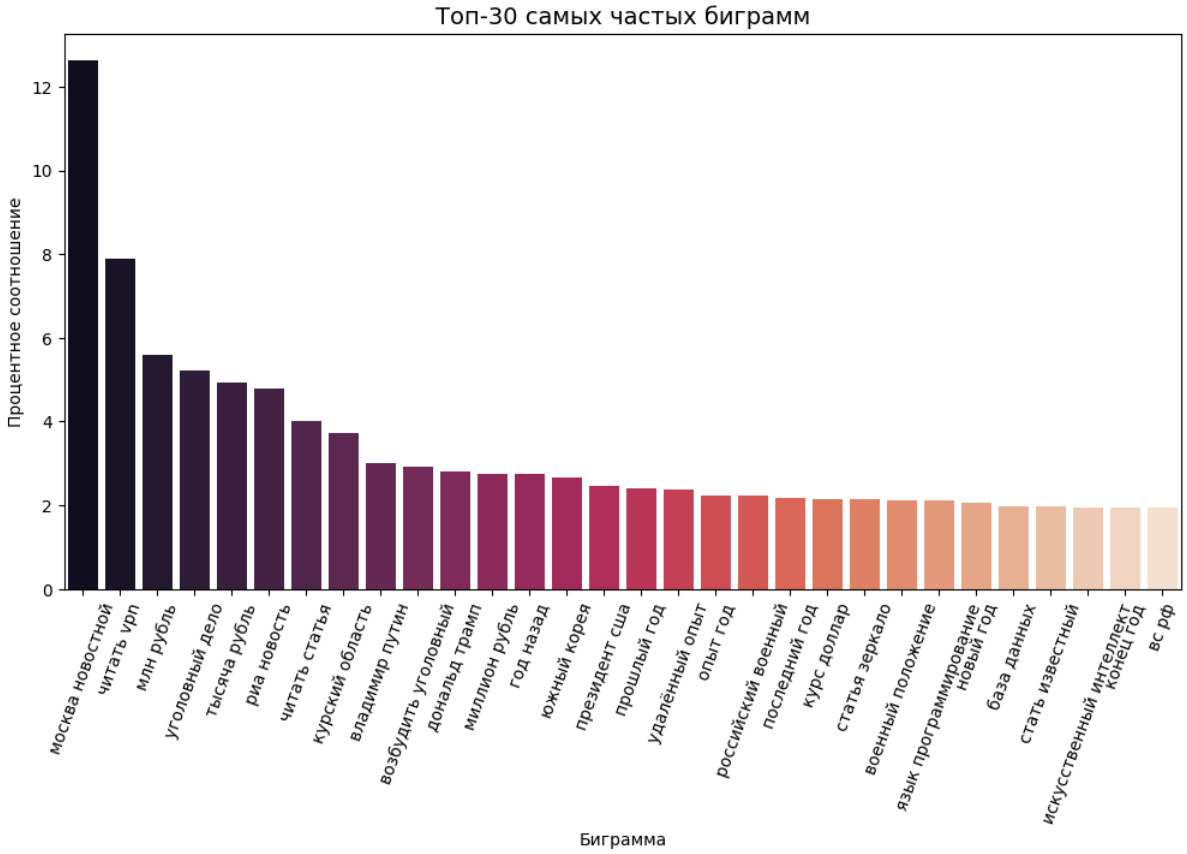


Рисунок 1.9 — График топ-30 самых частых биграмм в сообщениях

Как видно из графика 1.9, частотные биграммы чётко отражают ключевые категории. В финансовой тематике часто встречаются сочетания вроде «млн рублей», «тыс рублей», «курс доллара», что характерно для данной группы. В политических новостях и сообщениях о государственных структурах заметны биграммы «Владимир Путин», «Минобороны РФ», «ВС РФ», а также выражения «уголовное дело» и «возбуждено уголовное», которые могут встречаться и в личных темах при обсуждении криминальных событий.

Технологическая направленность категорий IT и науки проявляется в биграммах «data science», «читать VPN», «удалённо опыт» — последнее также может относиться к бизнес-тематике, например, в контексте удалённой работы. Упоминания «РИА Новости» и географические метки вроде «Курской области» чаще связаны с политикой или другими категориями при отсутствии чёткого контекста.

Рекламные биграммы представлены слабо, однако такие сочетания, как «статью зеркало» (возможные ссылки на альтернативные источники) и «кликнуть bitly», могут служить маркерами этой категории. Развлекательный контент и темы, связанные с играми, в выборке биграмм практически не представлены.

Для повышения качества классификации были удалены общие фразы (приложение А.4), такие как «стало известно» и «данный момент», поскольку они не способствуют разграничению категорий. Также были исключены биграммы вроде «читать статью» и «читать VPN», часто используемые каналами для указания первоисточника.

1.3.10 Анализ временной активности публикаций

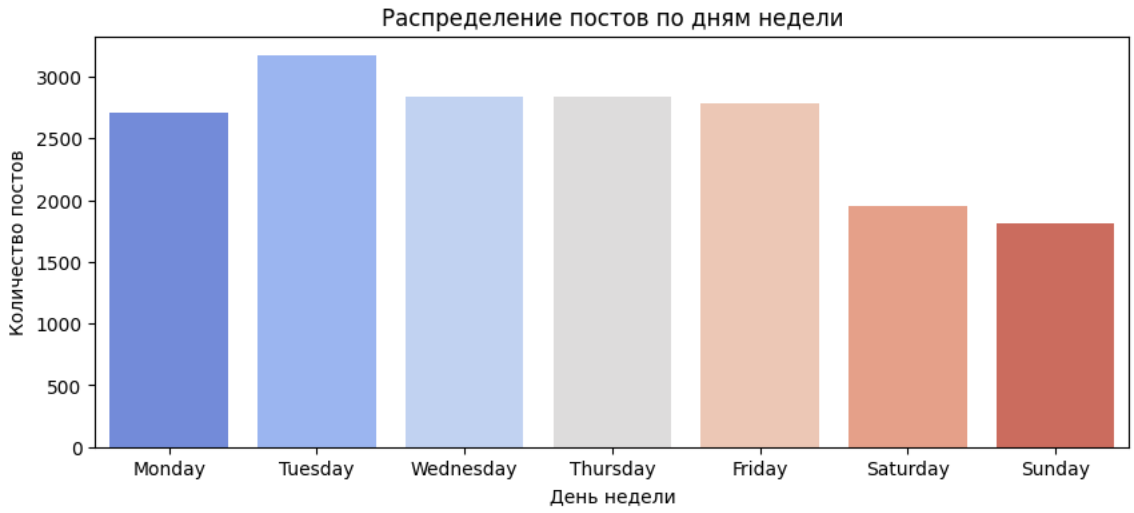


Рисунок 1.10 — График распределения постов по дням недели

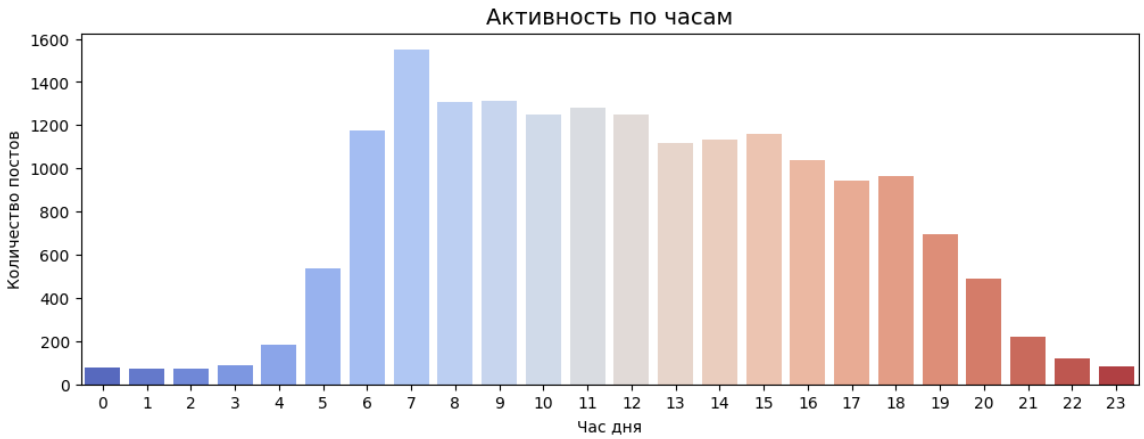


Рисунок 1.11 — График активности по часам



Рисунок 1.12 — График количества постов в день за все время

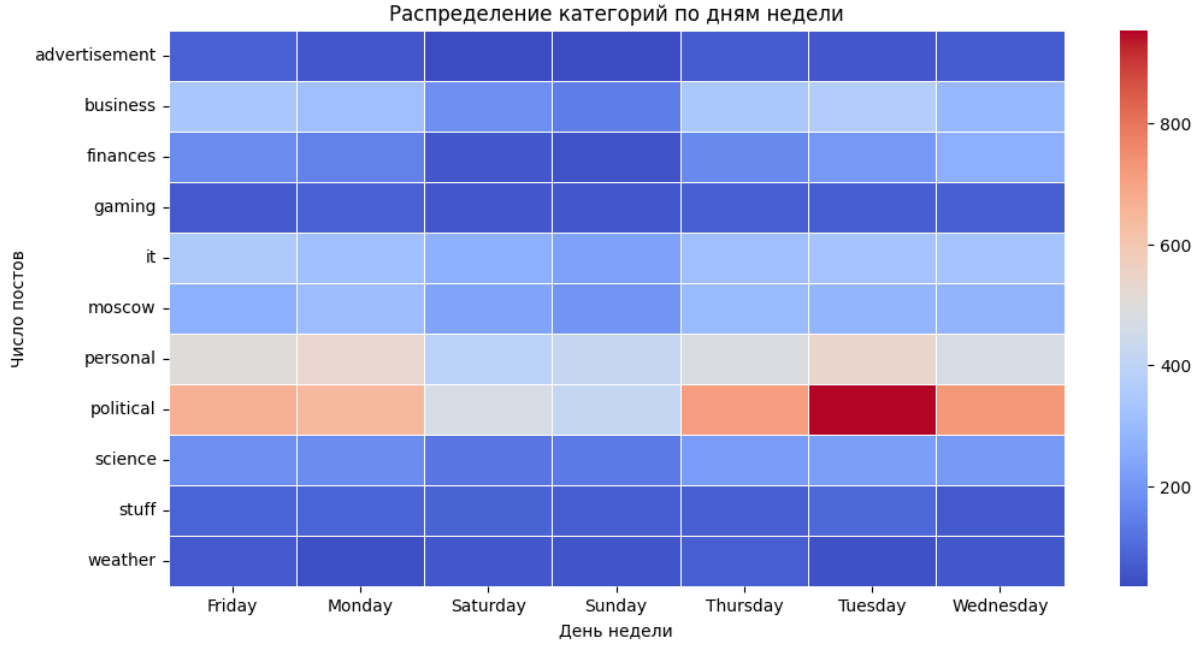


Рисунок 1.13 — График распределения категорий по дням недели

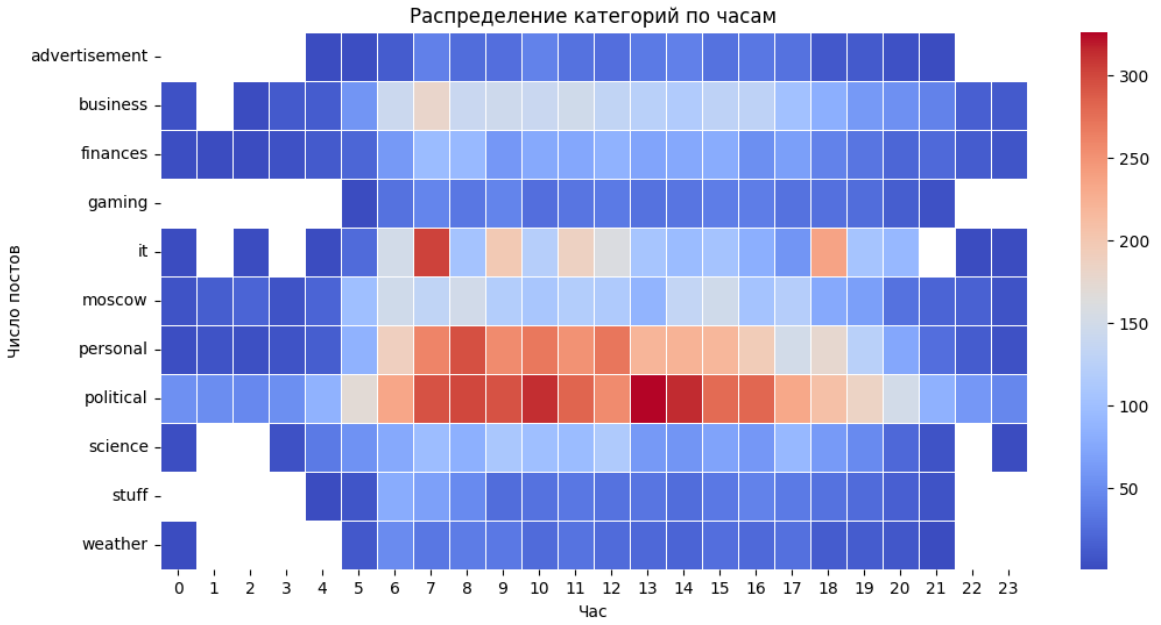


Рисунок 1.14 — График распределения категорий по часам

Анализ временной структуры данных, представленный на рисунке 1.12, показал, что посты охватывают период с октября 2023 по октябрь 2024 года, при этом пик активности приходится на октябрь 2024 года. Согласно графикам 1.11 и 1.13, наибольшая активность публикаций наблюдается в утренние и дневные часы — с 5 до 19 часов, с максимальным пиком около 7 утра, а чаще всего посты выходят по вторникам (рисунок 1.10). Это указывает на ориентированность публикаций на рабочие будни и утреннюю аудиторию.

Анализ тепловых карт и временных графиков (рисунки 1.10, 1.11, 1.13 и 1.14) демонстрирует, что время публикации играет важную роль в классификации контента: например, IT-тематика чаще публикуется в 7, 9 и 18 часов, тогда как политические новости — преимущественно с 7 до 10 и с 13 до 14 часов. При этом влияние дня недели на различие между классами выражено менее явно.

```
Kruskal-Wallis H-statistic: 40.373, p-value: 0.0000145649856
```

Рисунок 1.15 — Статистический тест Крускала–Уоллиса по часам

Дополнительно статистический тест Крускала–Уоллиса [7] (рисунок 1.15) подтвердил ($p < 0.005$) наличие значимых различий в распределении времени публикаций по категориям, что свидетельствует о существовании различных «временных паттернов жизни» контента в зависимости от тематики.

1.3.11 Визуализация категорий

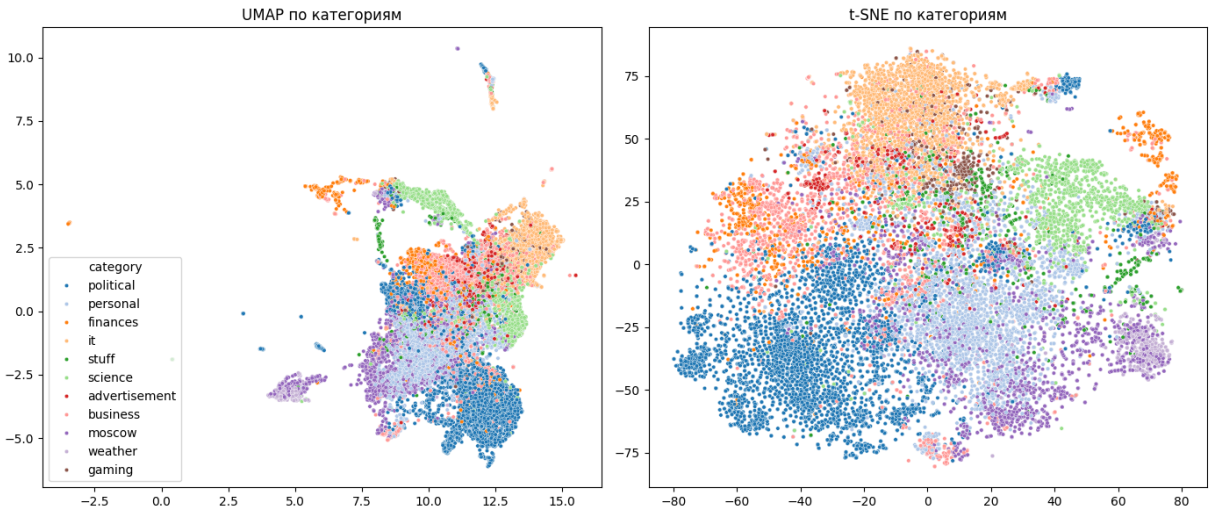


Рисунок 1.16 — Визуализация категорий через UMAP и t-SNE

UMAP и t-SNE [8], представленные на рисунке 1.16, показывают частичное разделение категорий: «political», «science», «personal», «finances» и «business» формируют чёткие кластеры, в то время как «weather» и «moscow» смешиваются, что указывает на их сходство [10]. Наличие точек-выбросов может говорить о возможных аномалиях в данных. В целом модель улавливает различия между

основными категориями, однако для спорных классов требуется доработка либо уточнение признаков.

1.3.12 Сравнение моделей LightGBM и RuBERT

Для подтверждения обоснованности выбора модели RuBERT [14] был проведён тест со сравнением с LightGBM [15] на ограниченной выборке (по 200 примеров на каждый класс). Цель эксперимента заключалась в доказательстве того, что смысловая составляющая текста является ключевым фактором в классификации, и что использование только простых лексических признаков недостаточно для точного разделения по тематикам.

Выбор пал на RuBERT [12], так как он является двунаправленным трансформером, что отличает его, например, от автогенеративных моделей вроде GPT [13]. Благодаря двунаправленной архитектуре, при обработке входной последовательности все токены могут использовать контекст друг друга, как слева, так и справа. Это критично для задач, где значение слова или фразы зависит от полного контекста предложения, что характерно для большинства публикаций в Telegram.

	precision	recall	f1-score	support
political	0.57	0.57	0.57	40
personal	0.57	0.50	0.53	40
finances	0.59	0.42	0.49	40
it	0.72	0.72	0.72	40
stuff	0.43	0.45	0.44	40
science	0.54	0.72	0.62	40
advertisement	0.61	0.62	0.62	40
business	0.38	0.30	0.33	40
moscow	0.49	0.68	0.57	40
weather	0.86	0.75	0.80	40
gaming	0.65	0.60	0.62	40
accuracy			0.58	440
macro avg	0.58	0.58	0.57	440
weighted avg	0.58	0.58	0.57	440

Рисунок 1.17 — Результат обученной модели LightGBM на тестовых данных

	precision	recall	f1-score	support
political	1.00	0.82	0.90	40
personal	0.64	0.68	0.66	40
finances	0.77	0.60	0.68	40
it	0.86	0.80	0.83	40
stuff	0.77	0.60	0.68	40
science	0.80	0.90	0.85	40
advertisement	0.59	0.88	0.71	40
business	0.60	0.62	0.61	40
moscow	0.76	0.72	0.74	40
weather	0.93	0.93	0.93	40
gaming	0.76	0.80	0.78	40
accuracy			0.76	440
macro avg	0.77	0.76	0.76	440
weighted avg	0.77	0.76	0.76	440

Рисунок 1.18 — Результат обученной модели RuBERT на тестовых данных

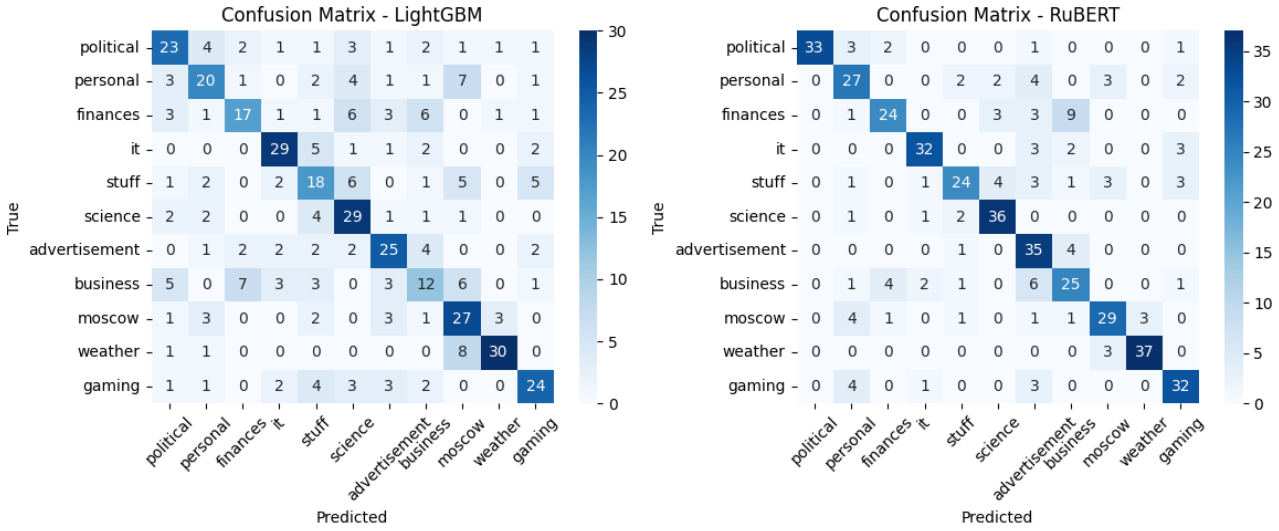


Рисунок 1.19 — Матрицы ошибок моделей

Модель LightGBM (приложение А.5) продемонстрировала среднюю точность на уровне 58%, где выделяются категории it с 72% и weather с 80%, а также категория business с самой низкой точностью в 33% (рисунок 1.17), тогда как RuBERT достиг 78% в среднем и можно заметить самый низкий результат в категории business, который составил 61%, остальные же составили точность от 68% до 93% (рисунок 1.18), что является существенным улучшением. Так же по матрицам ошибок на рисунке 1.19 видно разброс ответов у LightGBM, но и у RuBERT он тоже есть, но не настолько существенный. Этот результат подчёркивает высокую эффективность трансформеров в задаче текстовой

классификации, особенно в случаях, где необходимо учитывать контекст сообщений.

Полученные данные подтверждают, что модели на основе трансформеров, такие как RuBERT, обучающиеся на семантических связях внутри текста, значительно превосходят традиционные методы машинного обучения. В условиях, когда поверхностных признаков — таких как отдельные слова или время публикации — недостаточно для точного распознавания тематики, контекстуальное понимание становится критически важным для достижения высокой точности классификации.

1.3.13 Вывод

Анализ данных позволил обнаружить такие особенности, как наличие дубликатов, выраженный дисбаланс классов, различия в длине сообщений и характерные временные паттерны публикаций. Проведённая очистка и предобработка текста — включая удаление ссылок, стоп-слов, шаблонных фраз, а также лемматизацию — обеспечила качественную подготовку данных к машинному обучению.

Сравнительный анализ показал, что традиционные методы, такие как LightGBM, значительно уступают современным трансформерным архитектурам. Модель RuBERT, способная учитывать контекст и смысловую структуру текста, показала точность 78%, что существенно выше, чем 58% у LightGBM.

Для дальнейшего повышения качества классификации рекомендуется учитывать следующие аспекты: применять стратифицированное разбиение данных и кросс-валидацию (StratifiedKFold) для борьбы с дисбалансом классов, использовать F1-меру в качестве основной метрики качества модели, а также продолжать использование предобработанного текста и контекстно-зависимых моделей, таких как RuBERT.

Весь код, используемый для проведения анализа датасета, можно найти по ссылке: https://files.nktkln.com/University/EDA.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Постановка задачи обучения модели

Цель задачи — разработать модель машинного обучения, способную автоматически классифицировать текстовые сообщения из Telegram-каналов по заранее определённым тематическим категориям. Тип задачи — обучение с учителем для классификации текста, где каждому сообщению присваивается одна из нескольких заранее заданных меток.

На вход модели подаются текстовые сообщения из разных Telegram-каналов, предварительно очищенные и подготовленные, а также объединённые с информацией о времени публикации (час). На выходе модель должна выдавать метку, соответствующую тематической категории сообщения.

Для оценки качества модели будут использоваться метрики точности (accuracy), полноты (recall), точности (precision), F1-мера и ROC-AUC, что позволяет всесторонне оценить эффективность классификации [3]. В результате ожидается получить модель, способную с точностью более 80% классифицировать новые сообщения по заданным категориям.

2.2 Выбор и обоснование архитектуры

Основываясь на выводах анализа датасета из раздела 1.3.13, можно сделать вывод, что наилучшим выбором для нашей задачи является модель RuBERT, способная учитывать контекст и смысловую структуру текста, что особенно важно для точной классификации сообщений [13]. Анализ также показал, что для повышения качества модели следует применять стратифицированное разбиение данных и кросс-валидацию (StratifiedKFold) для борьбы с дисбалансом классов. В качестве метрик оценки рекомендуется использовать точность (accuracy), полноту (recall), точность (precision), F1-меру и ROC-AUC [3], что позволит всесторонне оценить эффективность классификации. При этом датасет будет

предварительно обработан в соответствии с выводами анализа: удаление ссылок, хештегов, элементов markdown-разметки, эмодзи, а также удаление стоп-слов из списка библиотеки NLTK [4] и преобразование слов в лемматизированную форму с помощью библиотеки SpaCy [5][6].

2.3 Процесс обучения

Перед началом обучения модели осуществляется загрузка и предобработка датасета (приложение В.1). После очистки составляются входные и выходные списки: входные данные представляют собой строки вида '[TEXT] ' + clean_text + ' [HOUR] ' + hour, а категории преобразуются в числовые идентификаторы. Далее датасет разбивается на обучающую и тестовую выборки с использованием стратифицированного разбиения из библиотеки scikit-learn, что позволяет сохранить пропорции классов [1] (приложение В.2). Затем обучающая выборка разделяется на k фолдов с помощью StratifiedKFold для дальнейшего кросс-валидационного обучения.

Само дообучение модели производится по такому алгоритму, где для каждого фолда производится: разбиение на обучающую и валидационную выборки. Данные преобразуются в формат Dataset из библиотеки datasets (приложение В.3) и производится токенизация текстов через встроенный токенеризатор RuBERT [19] (приложение В.4). Загружается «чистая» модель RuBERT, которая при наличии GPU переносится на устройство CUDA. Настраивается оптимизатор AdamW с планировщиком linear_with_warmup (приложение В.5) — этот подход широко используют для стабилизации обучения и ускорения сходимости, и запускается цикл обучения на заданное число эпох. Для каждой эпохи производится цикл из предоставляемых данных. На каждой итерации которого выполняется прямой проход, вычисляется функция потерь (loss), проводится обратное распространение ошибки, обновляются веса и корректируется скорость обучения (learning rate) (приложение В.4). После этого модель оценивается на валидационных данных (приложение В.6): вычисляется

точность, формируется отчёт, и при улучшении метрик текущая модель сохраняется как наилучшая [20] (приложение В.7). По завершении процесса обучения сохраняется модель с наилучшими показателями на валидационных данных. Затем эта модель загружается и проводится финальная оценка её качества на отложенной тестовой выборке (приложение В.8), что позволяет объективно проверить её обобщающую способность и применимость к ранее не встречавшимся данным.

Весь код, используемый для обучения модели, можно найти по ссылке: https://files.nktkln.com/University/EDA.

2.4 Проведение экспериментов и анализ результатов

После успешного обучения модели была проведена финальная оценка на основной тестовой выборке.

```
Test Accuracy: 0.9443

Classification Report:
               precision    recall  f1-score   support

     business       0.92      0.91      0.92       203
           it       0.94      0.98      0.96       241
     personal       0.95      0.95      0.95       339
      weather       0.93      0.95      0.94        40
       gaming       0.96      0.94      0.95        52
     finances       0.90      0.94      0.92       114
        stuff       0.90      0.93      0.92        61
    political       0.98      0.99      0.99       463
advertisement       0.89      0.91      0.90        44
      science       1.00      0.98      0.99       128
       moscow       0.96      0.94      0.95       192

    micro avg       0.95      0.96      0.95      1877
    macro avg       0.94      0.95      0.94      1877
 weighted avg       0.95      0.96      0.95      1877

Macro-Averaged ROC-AUC: 0.9969
```

Рисунок 2.1 — Результаты обученной модели на тестовой выборке

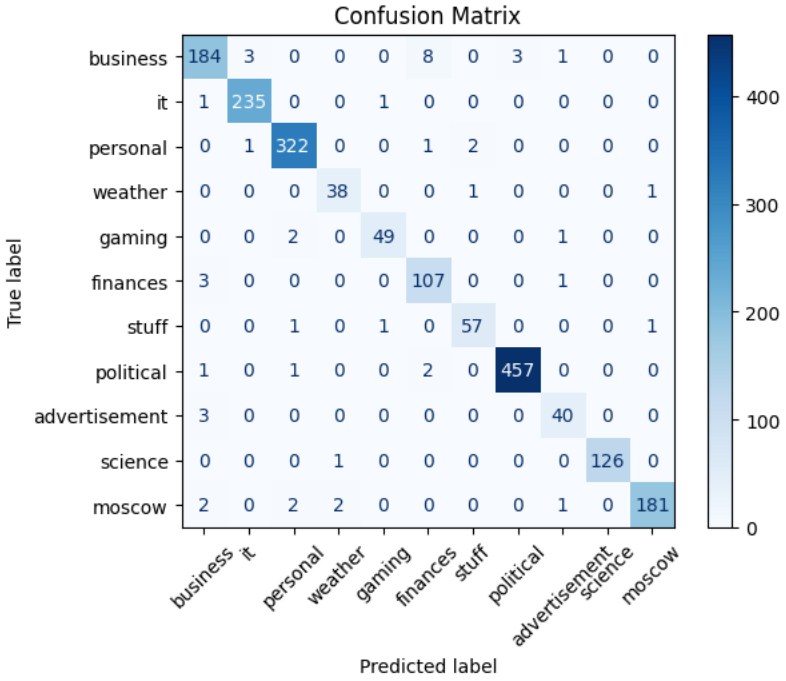


Рисунок 2.2 — Матрица ошибок обученной модели на тестовой выборке

Как показано на рисунке 2.1, модель достигла следующих результатов:

1. Accuracy (точность классификации): 0.9443
2. Macro-Averaged F1-score: 0.94
3. Macro-Averaged ROC-AUC: 0.9969

Как можно видеть из рисунков 2.1 и 2.2, модель продемонстрировала высокую устойчивость даже по слабо представленным категориям, таким как weather и advertisement, что свидетельствует о её способности справляться с умеренным дисбалансом классов. Наивысшие значения F1-мера зафиксированы для классов political (0.99), science (0.99) и it (0.96), что указывает на хорошую различимость этих тематик. Относительно низкие результаты отмечены для категорий advertisement (0.90) и finances (0.92), что может быть обусловлено схожестью лексики с другими классами и меньшим количеством обучающих примеров. Высокий показатель ROC-AUC (0.9969) подтверждает способность модели уверенно различать классы даже при вероятностной интерпретации. Высокие значения точности и F1-мера, особенно на фоне несбалансированного распределения классов, свидетельствуют о надёжности модели в условиях реального применения.

ЗАКЛЮЧЕНИЕ

В рамках настоящего исследования была решена задача автоматической тематической классификации сообщений из Telegram-каналов с использованием методов машинного обучения. Проведённый анализ предметной области подтвердил актуальность проблемы информационной перегрузки пользователей и обосновал необходимость создания интеллектуальных инструментов для фильтрации и категоризации контента.

На этапе подготовки данных была выполнена комплексная предобработка: удаление дубликатов, очистка текста от нерелевантных элементов, лемматизация и исключение стоп-слов. Анализ структуры выборки выявил значительный дисбаланс между классами и высокую вариативность длины сообщений, что было учтено при проектировании модели и выборе методик оценки.

Эксперименты показали, что традиционные алгоритмы машинного обучения (например, LightGBM) уступают трансформерным моделям по качеству классификации. Модель RuBERT была выбрана как наиболее подходящая для работы с русскоязычными текстами благодаря способности учитывать контекст и смысловую структуру сообщений. Обученная модель достигла точности 94,4%, а также показала высокое значение ROC-AUC (0.9969), что свидетельствует о её устойчивости и высокой способности к различению классов даже при наличии дисбаланса.

Таким образом, была успешно реализована и протестирована модель, способная эффективно классифицировать сообщения по тематическим категориям. Предложенное решение может применяться в системах персонализированной новостной ленты, рекомендательных сервисах и других приложениях, где требуется интеллектуальная обработка текстового контента.

Перспективы дальнейшего развития проекта включают:

1. расширение объёма и разнообразия обучающего датасета, особенно для недостаточно представленных категорий;

2. реализацию мультиклассового и мультиязычного подхода к классификации;

3. интеграцию модели в Telegram-бот или веб-интерфейс для взаимодействия с пользователями в реальном времени.

Полученные результаты подтверждают высокую эффективность современных моделей на базе трансформеров в задачах автоматической классификации текста и демонстрируют их применимость в реальных пользовательских сценариях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Dyakonov, A. Дисбаланс классов — Анализ малых данных. [Электронный ресурс] // Wordlim WordPress : блог. — URL: https://alexanderdyakonov.wordpress.com/2021/05/27/imbalance/ (дата обращения: 10.06.2025);

2. Борьба с несбалансированными данными. [Электронный ресурс] // Habr : корпоративный блог. — URL: https://habr.com/ru/companies/otus/articles/769242/ (дата обращения: 10.06.2025);

3. Методы решения проблемы дисбаланса классов в машинном обучении. [Электронный ресурс] // ITSTD Journal. — PDF. (2024). — URL: https://itstd-journal.ru/wp-content/uploads/2024/03/METHODS-FOR-SOLVING-THE-CLASS-IMBALANCE-PROBLEM-IN-.pdf (дата обращения: 10.06.2025);

4. Котюбеев, Р. 4 главных метода предобработки текста в NLP с Python / Р. Котюбеев // Python School : блог. — URL: https://python-school.ru/blog/nlp/nlp-text-preprocessing/ (дата обращения: 10.06.2025);

5. Васильев, Ю. Обработка естественного языка. Python и spaCy на практике / Ю. Васильев. — СПб.: Питер, 2021. — 256 с. — (Серия «Библиотека программиста»);

6. «Как нормализовать текст в spaCy и NLTK» [Электронный ресурс] // Python School : блог. — URL: https://python-school.ru/blog/nlp/text-normalization-spacy-vs-nltk/ (дата обращения: 10.06.2025);

7. SkillFactory. Критерий Краскела–Уоллиса: что это такое простыми словами. [Электронный ресурс] // SkillFactory: блог. — URL: https://blog.skillfactory.ru/glossary/kriterij-kraskela-uollisa/ (дата обращения: 10.06.2025);

8. Сравнительный анализ методов снижения размерности: PCA, t-SNE и UMAP / В. Иванов, И. Петров // Научные исследования и разработки. — 2022. — Т. 1, № 2. — С. 45–52. — URL:

https://www.dependability.ru/jour/article/view/496/784 (дата обращения: 10.06.2025);

9. Сравнительный анализ моделей векторизации текстов для задачи анализа тональности / Е. Сидоров, М. Иванова // ResearchGate : сайт. — URL: https://www.researchgate.net/publication/388062362_Sravnenie_modelej_vektorizacii_tekstov_dla_zadaci_analiza_tonalnosti_korotkih_soobsenij_iz_socialnyh_setej (дата обращения: 10.06.2025);

10. Сравнительный анализ точности методов визуализации многомерных данных при помощи алгоритмов снижения пространства признаков PCA и t-SNE / В. Петров, И. Сидорова // CyberLeninka : сайт. — URL: https://cyberleninka.ru/article/n/vizualizatsiya-mnogomernyh-naborov-dannyh-pri-pomoschi-algoritmov-snizheniya-prostranstva-priznakov-pca-i-t-sne (дата обращения: 10.06.2025);

11. Обзор трансформеров и их применения в обработке естественного языка. Зоопарк трансформеров: большой обзор моделей от BERT до GPT [Электронный ресурс] // Habr. — URL: https://habr.com/ru/companies/just_ai/articles/733110/ (дата обращения: 10.06.2025);

12. Адаптация BERT для русского языка. Линейка русскоязычных энкодеров ruRoPEBert и как она работает [Электронный ресурс] // Habr. — URL: https://habr.com/ru/companies/tochka/articles/797561/ (дата обращения: 10.06.2025);

13. RuBERT от DeepPavlov. Пять лучших NLP инструментов для работы с русским языком на Hugging Face [Электронный ресурс] // Habr. — URL: https://habr.com/ru/companies/otus/articles/808435/ (дата обращения: 10.06.2025);

14. СберУстройства. ruT5, ruRoBERTa, ruBERT: как мы обучили серию моделей для русского языка [Электронный ресурс] // Habr. — URL: https://habr.com/ru/companies/sberdevices/articles/567776/ (дата обращения: 10.06.2025);

15. Кузнецов, И. А. Определение пола автора текста в коллекции русских многожанровых текстов с искажениями с использованием моделей машинного обучения [Электронный ресурс] // ResearchGate. — URL: https://www.researchgate.net/publication/329778194_OPREDELENIE_POLA_AVTORA_TEKSTA_V_KOLLEKCII_RUSSKIH_MNOGOZANROVYH_TEKSTOV_S_ISKAZENIAMI_S_ISPOLZOVANIEM_MODELEJ_MASINNOGO_OBUCENIA (дата обращения: 10.06.2025);

16. Библиотека для извлечения данных и моделирования тематики сообщений Telegram — pytopicgram [Электронный ресурс] // GitHub : сайт. — URL: https://github.com/ugr-sail/pytopicgram (дата обращения: 10.06.2025);

17. Центральный хаб экосистемы TON: цены, биржи, кошельки, бриджи, DEX, NFT, приложения, сигналы, колы, аналитика и сообщества TON [Электронный ресурс] // tonfor.pro : сайт. — URL: https://tonfor.pro/ (дата обращения: 10.06.2025);

18. Дообучение ruBERT на специфическом корпусе для задачи классификации. [Электронный ресурс] // DeepPavlov Community Forum. — URL: https://forum.deeppavlov.ai/t/rubert/963 (дата обращения: 10.06.2025);

19. Fine-tuning моделей с использованием Trainer API. [Электронный ресурс] // Hugging Face LLM Course. — URL: https://huggingface.co/learn/llm-course/ru/chapter3/3 (дата обращения: 10.06.2025).

ПРИЛОЖЕНИЯ

Приложение А – Код из исследовательского анализа данных

Листинг А.1 — Часть, отвечающая за первоначальную очистку датасета.

```
# %%
df = pd.read_csv('all_channel_posts.csv', parse_dates=['date'])
df.head()

# %%
print(f"Размер датасета: {len(df)} постов")
print(f"Колонки: {df.columns.tolist()}")
df.info()

# %%
print(f"Пропуски в данных:\n{df.isnull().sum()}")
print(f"\nПолные дубликаты: {df.duplicated().sum()}")
print(f"Дубликаты текста: {df['text'].duplicated().sum()}")
print(f"Дубликаты текста с категориями: {df.duplicated(subset=['text',
'category']).sum()}")

# %%
df = df.drop_duplicates(subset=['text'])
df = df[df['category'] != 'other']
df = df.reset_index(drop=True)
```

Листинг А.2 — Часть, отвечающая за предобратку текстовых данных.

```
# %%
nltk.download('stopwords')

stop = set(stopwords.words('russian'))
nlp = spacy.load('ru_core_news_sm')

def clean_text_symbols(text: str) -> str:
    text = re.sub(r"(https?://|www\.)\S+|bit\.ly/\S+|t\.co/\S+", "", text)
    text = re.sub(r"#[\wа-яА-ЯёЁ]+", "", text)
    text = re.sub(r"@[\wа-яА-ЯёЁ]+", "", text)
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"(\*\*|__)(.*?)\1", r"\2", text)
    text = re.sub(r"(\*|_)(.*?)\1", r"\2", text)
    text = re.sub(r"~~(.*?)~~", r"\1", text)
    text = re.sub(r"`{1,2}(.*?)`{1,2}", r"\1", text)
    text = re.sub(r"#{1,6}\s*", "", text)
    text = re.sub(r"\[(.*?)\]\(.*?\)", r"\1", text)
    text = re.sub(r"!\[.*?\]\(.*?\)", "", text)
    text = re.sub(r">\s*", "", text)
    text = re.sub(r"[^a-zA-Za-яА-ЯёЁ\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def remove_stop_words(text: str, stop: set) -> str:
    text = " ".join([word for word in text.split() if word not in stop])
    return text

def lemmatize_text(text: str) -> str:
    doc = nlp(text)
    lemmas = [token.lemma_ for token in doc]
    return ' '.join(lemmas)
```

Продолжение листинга А.2.

```
def preprocess_text(text: str, stop: set) -> str:
    text = text.lower()
    text = clean_text_symbols(text)
    text = lemmatize_text(text)
    text = remove_stop_words(text, stop)
    return text

df['clean_text'] = df['text'].apply(lambda x: preprocess_text(x, stop))

# %%
def get_last_text_line(text: str) -> str:
    if len(text.split('\n')) == 1:
        return ""
    return text.strip().split('\n')[-1]

def remove_frequent_last_line(text: str, frequent_lines: list) -> str:
    lines = text.strip().split('\n')
    if len(lines) > 1 and lines[-1] in frequent_lines:
        lines = lines[:-1]
    return '\n'.join(lines).strip()

last_lines_series = df['clean_text'].apply(get_last_text_line)
last_lines_series = last_lines_series.replace("", pd.NA).dropna()
last_line_frequencies = last_lines_series.value_counts()
frequent_last_lines = last_line_frequencies[last_line_frequencies >
1].index.tolist()

df['clean_text'] = df['clean_text'].apply(lambda x: remove_frequent_last_line(x,
frequent_last_lines))
```

Листинг А.3 — Часть с дополнительной очисткой данных.

```
# %%
stop.update(["это", "также", "подписаться", "который", "изза", "прислать",
"ещё", "которые", "топор", "подписывайся", "mash", "shot", "live", "новостную",
"кибертопор"])

df['clean_text'] = df['clean_text'].apply(lambda x: remove_stop_words(x, stop))
```

Листинг А.4 — Часть с дополнительной очисткой данных от биграмм.

```
# %%
bigrams_to_remove = {
    "стало известно",
    "данный момент",
    "читать статью",
    "читать vpn",
    "риа новости"
}

def remove_bigrams(text: str, bigrams_to_remove: set) -> str:
    for bigram in bigrams_to_remove:
        if bigram in text:
            text = text.replace(bigram, "")
    return text.strip()

df['clean_text'] = df['clean_text'].apply(lambda x: remove_bigrams(x,
bigrams_to_remove))
```

Листинг А.5 — Часть с моделью LightGBM.

```
# %%
unique_labels = df['category'].unique()
label_to_id = {label: idx for idx, label in enumerate(unique_labels)}
id_to_label = {idx: label for label, idx in label_to_id.items()}
target_names = [id_to_label[i] for i in sorted(id_to_label)]
df['category_id'] = df['category'].apply(lambda x: label_to_id[x])

df_small_balanced = df.groupby('category').sample(n=200, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    df_small_balanced[['clean_text', 'hour']], df_small_balanced['category_id'],
test_size=0.2, random_state=42, stratify=df_small_balanced['category_id']
)

# %%
tfidf = TfidfVectorizer(max_features=5000)
X_train_tfidf = tfidf.fit_transform(X_train['clean_text'])
X_val_tfidf = tfidf.transform(X_val['clean_text'])

X_train_final = hstack([X_train_tfidf, X_train['hour'].values.reshape(-1, 1)])
X_val_final = hstack([X_val_tfidf, X_val['hour'].values.reshape(-1, 1)])

lgb_train = lgb.Dataset(X_train_final, label=y_train)
lgb_eval = lgb.Dataset(X_val_final, label=y_val, reference=lgb_train)

params = {
    'objective': 'multiclass',
    'num_class': len(df_small_balanced['category_id'].unique()),
    'metric': 'multi_logloss',
    'boosting_type': 'gbdt',
    'learning_rate': 0.1,
    'num_leaves': 31,
    'verbosity': -1,
    'seed': 42
}

model_lgb = lgb.train(
    params,
    lgb_train,
    valid_sets=[lgb_train, lgb_eval],
    num_boost_round=100,
    callbacks=[lgb.early_stopping(stopping_rounds=10)]
)

# %%
predictions_lgb = model_lgb.predict(X_val_final)
predicted_labels_lgb = predictions_lgb.argmax(axis=1)
print(classification_report(y_val, predicted_labels_lgb,
target_names=target_names))
```

Листинг А.6 — Часть с моделью RuBERT.

```
# %%
class Dataset(torch.utils.data.Dataset):
    def __init__(self, texts, hours, labels, tokenizer, max_len=128):
        self.texts = texts
        self.hours = hours
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding='max_length',
            max_length=self.max_len,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].squeeze(),
            'attention_mask': encoding['attention_mask'].squeeze(),
            'numeric_features': torch.tensor(self.hours[idx],
dtype=torch.float),
            'labels': torch.tensor(self.labels[idx], dtype=torch.long)
        }

class CustomModel(torch.nn.Module):
    def __init__(self, model_name, num_labels):
        super().__init__()
        self.bert =
AutoModelForSequenceClassification.from_pretrained(model_name,
num_labels=num_labels).bert
        self.classifier = torch.nn.Linear(self.bert.config.hidden_size + 1,
num_labels)
        self.num_labels = num_labels

    def forward(self, input_ids, attention_mask, numeric_features, labels=None):
        outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask)
        pooled_output = outputs.pooler_output
        combined = torch.cat((pooled_output, numeric_features.unsqueeze(1)),
dim=1)
        logits = self.classifier(combined)
        loss = None
        if labels is not None:
            loss_fct = torch.nn.CrossEntropyLoss()
            loss = loss_fct(logits.view(-1, self.num_labels), labels.view(-1))
        return {'loss': loss, 'logits': logits}

tokenizer = AutoTokenizer.from_pretrained("DeepPavlov/rubert-base-cased")
train_dataset = Dataset(X_train['clean_text'].tolist(),
X_train['hour'].tolist(), y_train.tolist(), tokenizer)
val_dataset = Dataset(X_val['clean_text'].tolist(), X_val['hour'].tolist(),
y_val.tolist(), tokenizer)
```

Продолжение листинга А.6.

```
model_rubert = CustomModel("DeepPavlov/rubert-base-cased",
num_labels=len(df_small_balanced['category_id'].unique()))
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    logging_dir='./logs',
    do_eval=True,
    save_total_limit=1
)

def compute_metrics(pred):
    logits, labels = pred
    preds = logits.argmax(axis=-1)
    accuracy = (preds == labels).mean()
    f1 = f1_score(labels, preds, average="macro")
    return {"accuracy": accuracy, "f1": f1}

trainer = Trainer(
    model=model_rubert,
    args=training_args,
```

Продолжение листинга А.6.

```
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics
)

trainer.train()

# %%
predictions_rubert = trainer.predict(val_dataset)
predicted_labels_rubert = predictions_rubert.predictions.argmax(axis=-1)
true_labels = predictions_rubert.label_ids

print(classification_report(true_labels, predicted_labels_rubert,
target_names=target_names))
```

Приложение Б – Исходный код скрипта парсинга Telegram-каналов

Листинг Б.1 — Исходный код скрипта для извлечения сообщений из Telegram-каналов.

```
import os
import json
import logging
from typing import List, Dict

from telethon.tl.types import Message
from telethon.sync import TelegramClient
from telethon.tl.custom.dialog import Dialog

# Replace with your API ID and API Hash
API_ID = 'your_api_id'  # Replace with your actual API ID
API_HASH = 'your_api_hash'  # Replace with your actual API Hash

# Logging configuration
logging.basicConfig(level=logging.INFO)
```

Продолжение листинга Б.1.

```
logger = logging.getLogger(__name__)

# Constants
OUTPUT_FILE_TEMPLATE = "{}_messages.json"
MESSAGE_LIMIT = 3000  # Default message limit


async def fetch_channel_messages(client: TelegramClient, channel: Dialog) ->
List[Dict]:
    """
    Fetch messages from a specific Telegram channel.

    :param client: Instance of the Telegram client.
    :param channel: The channel from which to fetch messages.
    :returns: A list of dictionaries containing message details.
    """
    logger.info(f"Fetching the last {MESSAGE_LIMIT} messages from channel:
{channel.name}")
    messages = []

    async for message in client.iter_messages(channel.id, limit=MESSAGE_LIMIT):
        if isinstance(message, Message) and message.text:
            messages.append({
                'message_id': message.id,
                'sender_id': message.sender_id,
                'text': message.text,
                'date': message.date.isoformat(),
                'channel': channel.name
            })

    logger.info(f"Fetched {len(messages)} messages from channel:
{channel.name}")
    return messages


def save_messages_to_file(messages: List[Dict], channel_id: int) -> None:
    """
    Save messages to a JSON file.

    :param messages: List of message dictionaries to save.
    :param channel_id: ID of the channel (used for file naming).
    """
    if not messages:
        logger.warning("No messages to save.")
        return

    file_path = OUTPUT_FILE_TEMPLATE.format(abs(channel_id))

    # Load existing messages if the file exists
    if os.path.exists(file_path):
        with open(file_path, 'r', encoding='utf-8') as file:
            existing_messages = json.load(file)
        messages.extend(existing_messages)

    # Save all messages to the file
    with open(file_path, 'w', encoding='utf-8') as file:
        json.dump(messages, file, ensure_ascii=False, indent=4)

    logger.info(f"Messages saved to {file_path}")
```

Продолжение листинга Б.1.

```
async def process_channels(client: TelegramClient) -> None:
    """
    Process all channels the user has access to and fetch messages.

    :param client: Instance of the Telegram client.
    """
    await client.start()
    channels = [dialog for dialog in await client.get_dialogs() if
dialog.is_channel]

    for channel in channels:
        logger.info(f"Processing channel: {channel.name}")
        messages = await fetch_channel_messages(client, channel)
        save_messages_to_file(messages, channel.id)


def main() -> None:
    client = TelegramClient('telegram_message_exporter', API_ID, API_HASH)
    client.loop.run_until_complete(process_channels(client))


if __name__ == '__main__':
    main()
```

Приложение В – Исходный код обучения модели

Листинг В.1 — Часть кода, отвечающая за загрузку и предобработку датасета.

```
# %%
# Loading the data
file_path = "all_channel_posts.csv"  # Provide the path to your CSV file
df = pd.read_csv('all_channel_posts.csv', parse_dates=['date'])

df = df.drop_duplicates(subset=['text'])
df = df[df['category'] != 'other']
df = df.reset_index(drop=True)

# %%
nltk.download('stopwords')

stop = set(stopwords.words('russian'))
nlp = spacy.load('ru_core_news_sm')

def clean_text_symbols(text: str) -> str:
    text = re.sub(r"(https?://|www\.)\S+|bit\.ly/\S+|t\.co/\S+", "", text)
    text = re.sub(r"#[\wа-яА-ЯёЁ]+", "", text)
    text = re.sub(r"@[\wа-яА-ЯёЁ]+", "", text)
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"(\*\*|__)(.*?)\1", r"\2", text)
    text = re.sub(r"(\*|_)(.*?)\1", r"\2", text)
    text = re.sub(r"~~(.*?)~~", r"\1", text)
    text = re.sub(r"`{1,2}(.*?)`{1,2}", r"\1", text)
    text = re.sub(r"#{1,6}\s*", "", text)
    text = re.sub(r"\[(.*?)\]\(.*?\)", r"\1", text)
    text = re.sub(r"!\[.*?\]\(.*?\)", "", text)
    text = re.sub(r">\s*", "", text)
    text = re.sub(r"[^a-zA-Zа-яА-ЯёЁ0-9\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

Продолжение листинга В.1.

```
def remove_stop_words(text: str, stop: set) -> str:
    text = " ".join([word for word in text.split() if word not in stop])
    return text

def lemmatize_text(text: str) -> str:
    doc = nlp(text)
    lemmas = [token.lemma_ for token in doc]
    return ' '.join(lemmas)

def preprocess_text(text: str, stop: set) -> str:
    text = text.lower()
    text = clean_text_symbols(text)
    text = lemmatize_text(text)
    text = remove_stop_words(text, stop)
    return text

df['clean_text'] = df['text'].apply(lambda x: preprocess_text(x, stop))

# %%
def get_last_text_line(text: str) -> str:
    if len(text.split('\n')) == 1:
        return ""
    return text.strip().split('\n')[-1]

def remove_frequent_last_line(text: str, frequent_lines: list) -> str:
    lines = text.strip().split('\n')
    if len(lines) > 1 and lines[-1] in frequent_lines:
        lines = lines[:-1]
    return '\n'.join(lines).strip()

last_lines_series = df['clean_text'].apply(get_last_text_line)
last_lines_series = last_lines_series.replace("", pd.NA).dropna()
last_line_frequencies = last_lines_series.value_counts()
frequent_last_lines = last_line_frequencies[last_line_frequencies >
1].index.tolist()

df['clean_text'] = df['clean_text'].apply(lambda x: remove_frequent_last_line(x,
frequent_last_lines))

# %%
stop.update(["это", "также", "подписаться", "который", "изза", "прислать",
"ещё", "которые", "топор", "подписывайся", "mash", "shot", "live", "новостную",
"кибертопор"])

df['clean_text'] = df['clean_text'].apply(lambda x: remove_stop_words(x, stop))

# %%
bigrams_to_remove = {
    "стало известно",
    "данный момент",
    "читать статью",
    "читать vpn",
    "риа новости"
}

def remove_bigrams(text: str, bigrams_to_remove: set) -> str:
    for bigram in bigrams_to_remove:
        if bigram in text:
            text = text.replace(bigram, "")
    return text.strip()
```

Продолжение листинга В.1.

```
df['clean_text'] = df['clean_text'].apply(lambda x: remove_bigrams(x,
bigrams_to_remove))

# %%
df['hour'] = df['date'].dt.hour # <-
```

Листинг В.2 — Часть кода, отвечающая за разбиение на тестовую и обучающую выборку.

```
# %%
# Extracting text and category labels
texts = ('[TEXT] ' + df['clean_text'] + ' [HOUR] ' +
df['hour'].astype(str)).tolist()
labels = df['category'].tolist()

# Converting labels to numerical identifiers
# unique_labels = list(set(labels)).sort()
unique_labels = ['business', 'it', 'personal', 'weather', 'gaming', 'finances',
'stuff', 'political', 'advertisement', 'science', 'moscow']
label_to_id = {label: idx for idx, label in enumerate(unique_labels)}
id_to_label = {idx: label for label, idx in label_to_id.items()}
labels = [label_to_id[label] for label in labels]

# Splitting data into training and testing sets
train_texts, test_texts, train_labels, test_labels = train_test_split(
    texts, labels, test_size=0.1, random_state=42, stratify=labels
)

# %%
# Setting up cross-validation
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Loading the pre-trained tokenizer and setting up the device
model_name = "DeepPavlov/rubert-base-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Листинг В.3 — Функция подготовки данных для модели.

```
# %%
# Preparing data for training and validation
def prepare_data(texts: list, labels: list, fold_idx: int, kfold:
StratifiedKFold) -> tuple:
    """
    Prepares the training and validation datasets for the given fold iteration.

    :param texts: List of input texts.
    :param labels: List of labels.
    :param fold_idx: Index of the current fold.
    :param kfold: StratifiedKFold object for cross-validation.
    :returns: A tuple containing:
            - train_data: A Dataset object for the training data.
            - val_data: A Dataset object for the validation data.
    """
    train_idx, val_idx = list(kfold.split(texts, labels))[fold_idx]

    train_texts = [texts[i] for i in train_idx]
    train_labels = [labels[i] for i in train_idx]
    val_texts = [texts[i] for i in val_idx]
    val_labels = [labels[i] for i in val_idx]

    train_data = Dataset.from_dict({"text": train_texts, "label": train_labels})
    val_data = Dataset.from_dict({"text": val_texts, "label": val_labels})

    train_data = train_data.map(tokenize_function, batched=True)
    val_data = val_data.map(tokenize_function, batched=True)
```

Продолжение листинга В.3.

```
    train_data.set_format(type="torch", columns=["input_ids", "attention_mask",
"label"])
    val_data.set_format(type="torch", columns=["input_ids", "attention_mask",
"label"])

    return train_data, val_data
```

Листинг В.4 — Код отвечающий за загрузку токенеризатора и функции токенеризации текста.

```
# %%
# Loading the pre-trained tokenizer and setting up the device
model_name = "DeepPavlov/rubert-base-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# %% [markdown]
# ## Tokenization Function

# %%
# Tokenization function
def tokenize_function(examples: dict) -> dict:
    """
    Tokenizes input texts using the pre-trained tokenizer.

    :param examples: A dictionary containing the texts to be tokenized.
    :returns: A dictionary containing tokenized texts.
    """
    return tokenizer(examples["text"], padding="max_length", truncation=True,
max_length=128)
```

Листинг В.5 — Основной цикл обучения модели.

```
# %%
# Training parameters
epochs = 3
batch_size = 8
learning_rate = 5e-5

# Global variables
best_accuracy = 0.0

# Training loop for each fold in cross-validation
for fold_idx in range(kfold.n_splits):
    print(f"\n{Fore.CYAN}Training fold {Style.RESET_ALL}{fold_idx + 1}")

    # Preparing data for training and validation
    train_data, val_data = prepare_data(train_texts, train_labels, fold_idx,
kfold)

    # Loading the model for each fold
    model = BertForSequenceClassification.from_pretrained(model_name,
num_labels=12)
    model.to(device)

    # Setting up optimizer and scheduler
    optimizer = AdamW(model.parameters(), lr=learning_rate)
    num_training_steps = epochs * len(train_data) // batch_size
    lr_scheduler = get_scheduler(
        name="linear",
        optimizer=optimizer,
        num_warmup_steps=0,
        num_training_steps=num_training_steps
```

Продолжение листинга В.5.

```
    )

    # DataLoader for training and validation
    train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_data, batch_size=batch_size)

    # Training the model over several epochs
    for epoch in range(epochs):
        print(f"{Fore.YELLOW}Epoch {Style.RESET_ALL}{epoch + 1}/{epochs}")
        avg_loss = train_model(model, train_loader, optimizer, lr_scheduler)
        print(f"{Fore.RED}Training loss: {Style.RESET_ALL}{avg_loss:.4f}")

    # Evaluating accuracy on validation data
    all_preds, all_labels, _ = evaluate_model(model, val_loader, device)
    accuracy = accuracy_score(all_labels, all_preds)
    print(f"{Fore.MAGENTA}Validation Accuracy: {Style.RESET_ALL}{accuracy:.4f}")

    # Saving the model if accuracy improved
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        save_model("best_model", model, tokenizer)

# Clearing CUDA cache if using GPU
if torch.cuda.is_available():
    torch.cuda.empty_cache()
```

Листинг В.6 — Функция оценки обученной модели.

```
# %%
# Model training function
def train_model(model: torch.nn.Module, train_loader: DataLoader, optimizer:
torch.optim.Optimizer, scheduler: torch.optim.lr_scheduler.LambdaLR) -> float:
    """
    Trains the model for one epoch.

    :param model: The model to be trained.
    :param train_loader: DataLoader for the training set.
    :param optimizer: The optimizer for the model.
    :param scheduler: Learning rate scheduler.
    :returns: The average loss for the epoch.
    """
    model.train()
    total_loss = 0
    for batch in tqdm(train_loader, desc="Обучение"):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)

        optimizer.zero_grad()
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        total_loss += loss.item()

        loss.backward()
        optimizer.step()
        scheduler.step()

    avg_loss = total_loss / len(train_loader)
    return avg_loss
```

Листинг В.7 — Функция для сохранении модели.

```
# %%
# Function to save the best model
def save_model(model_path: str, model: torch.nn.Module, tokenizer:
AutoTokenizer) -> None:
    """
    Saves the model and tokenizer to disk.

    :param model_path: Path to the directory where the model and tokenizer will
be saved.
    :param model: The trained model.
    :param tokenizer: The tokenizer used with the model.
    """
    model.save_pretrained(model_path)
    tokenizer.save_pretrained(model_path)
    print(f"{Fore.GREEN}Model saved at {Style.RESET_ALL}{model_path}")
```

Листинг В.8 — Часть кода с загрузкой лучшей модели и последующей её оценки.

```
# %%
# Function to load the best model
def load_model(model_path: str) -> tuple:
    """
    Loads the saved best model and tokenizer from the specified path.

    :param model_path: Path to the directory containing the saved model and
tokenizer.
    :returns: The model and tokenizer, if available, otherwise None.
    """
    if not os.path.exists(model_path):
        raise FileNotFoundError(f"Model path {model_path} does not exist.")

    model = BertForSequenceClassification.from_pretrained(model_path)
    tokenizer = AutoTokenizer.from_pretrained(model_path)
    print(f"{Fore.GREEN}Model and tokenizer loaded from
{Style.RESET_ALL}{model_path}")
    return model, tokenizer

# %%
# Loading the best model after training
model, tokenizer = load_model("best_model")
model.to(device)

# Preparing the test dataset for final evaluation
test_data = prepare_test_data(test_texts, test_labels)
test_loader = DataLoader(test_data, batch_size=batch_size)

# Final evaluation on the test set
all_preds, all_labels, all_probs = evaluate_model(model, test_loader, device)

# %%
# Printing final evaluation results
accuracy = accuracy_score(all_labels, all_preds)
print(f"{Fore.MAGENTA}Test Accuracy: {Style.RESET_ALL}{accuracy:.4f}\n")

# Unique classes present in the test set
present_classes = sorted(set(all_labels))

# Printing classification report
print(f"{Fore.MAGENTA}Classification Report:{Style.RESET_ALL}")
print(classification_report(
    all_labels,
    all_preds,
    labels=present_classes,
```

Продолжение листинга В.8.

```
    target_names=[id_to_label[idx] for idx in present_classes]
))

if len(set(all_labels)) == len(set(all_preds)):
    all_labels_ovr = np.zeros((len(all_labels), len(unique_labels)))
    for i, label in enumerate(all_labels):
        all_labels_ovr[i][label] = 1

    all_probs = np.array(all_probs)
    roc_auc = roc_auc_score(all_labels_ovr, all_probs, average="macro",
multi_class="ovr")
    print(f"{Fore.MAGENTA}Macro-Averaged ROC-AUC:
{Style.RESET_ALL}{roc_auc:.4f}")

# %%
# Generating and displaying the confusion matrix
present_classes = sorted(set(all_labels).union(set(all_preds)))
cm = confusion_matrix(all_labels, all_preds, labels=present_classes)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=[id_to_label[idx] for idx in present_classes])

disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix")
plt.show()
```